55TH B2GM · KEK

LLM activities for computing at KEK

Dhiraj Kalita · KEK IPNS, Belle II Core Computing (B2CC) · 55th B2GM · KEK

LLMs for Belle II computing operations

A glimpse of the prototypes the computing group is building to support our users, shifters and experts — work in progress, not a finished product.

GOAL 01

Less burden on users, shifters & experts

Ask in plain words. The agent searches the docs and queries the live systems — Rucio, DIRAC, FTS, k3s — instead of a person digging by hand.

GOAL 02

From issue to fix, faster

Shorten the time to spot a problem, find its cause and know the fix — and every answer points back to where it came from.

GOAL 03

gbasf2 & KEK internal docs, searchable

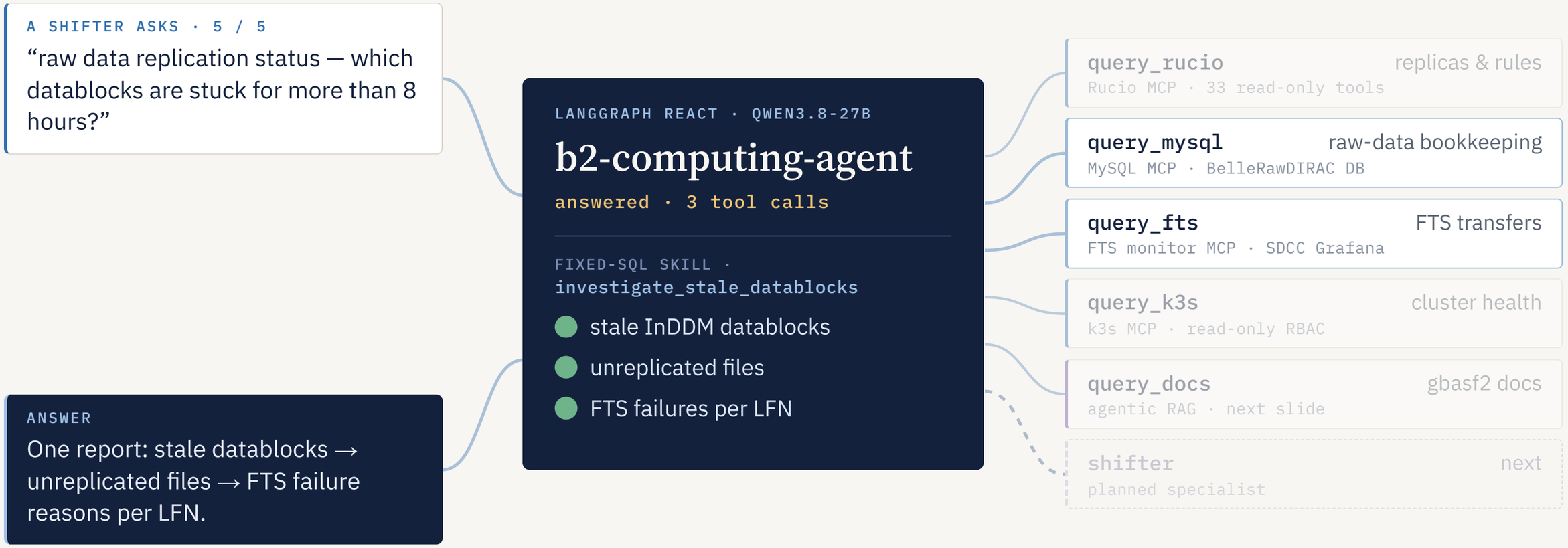
The gbasf2 and KEK internal docs are chunked and indexed, so answers come from the docs with citations.

Next: testing the **basf2 docs** — under discussion with the software group.

SCOPE Computing operations only. Everything shown is a **prototype**.

The agentic operations assistant

An orchestrator agent reads the question and routes it to specialist tools. Each specialist is its own small agent with a prompt file and one read-only MCP server.



The documentation assistant (Hybrid RAG)

Zooming into one tool from the last slide: how it answers a gbasf2 question from the docs — or says it does not know.

Key decision: look it up in the docs.

The gbasf2 docs → Cut into short passages (chunks), ready to search

Docs change? We rebuild the library — that is the whole update.

YOU ASK

A question in plain words

“how to generate additional replica of my analysis data”

REWORDED BY THE ASSISTANT

“gbasf2 command to replicate a dataset to another storage element”

LOOK IT UP

Find passages

- Replica management
- Replication status
- Storage elements
- Downloading datasets
- Listing datasets

up to 12, by keyword and by meaning (5 shown)

NARROW DOWN

Keep the best 3

- Replica managem... 0.96
- Replication status 0.88
- Storage elements 0.52

each scored 0–1 by a second model

CHECK

Good enough?

Is the best passage a strong enough match to answer from?

best 0.96 · needs 0.9

ENOUGH

YES

Answer, with sources

every claim points to the page it came from

STILL WEAK

“The docs don't cover this”

it says what is missing instead of guessing

Not enough? The assistant rewords it, looks again.

Raw data monitoring

1 · REPLICATION RULES OF A RAW DATABLOCK

You

can you check the rule id of the dataset /belle/Raw/e0042/physics/r01976/sub00

b2-computing-agent

The dataset `/belle/Raw/e0042/physics/r01976/sub00` has **5 replication rules**, all in state **OK** (24/24 locks satisfied):

RULE ID	ACCOUNT	STATE	RSE EXPRESSION	COPIES	LOCKED	ACTIVITY
`221bbb2116994d7c98083dd7e29b80c7`	`belle_raw`	OK	`KEK-RAWTMP-SE`	1	no	Raw Export
`819652c34e654f399c19bf60f368fe97`	`belleraw1`	OK	`ANY=true`	1	no	default
`c4d1e45b4cf64bdb9f8282022b656caa`	`ruciodm`	OK	`BNL-TAPE-SE`	1	yes	Raw Export
`086c274cb9b047988e3d1c4620d81c6f`	`ruciodm`	OK	`BNL-TMP-SE`	1	no	Raw Export
`ad191644330a4fbf9482f54e82a0be33`	`ruciodm`	OK	`KEK-RAW-SE`	1	yes	Raw Export

Notes:

- Four rules are subscription-driven; the `belleraw1` rule (`ANY=true`) is a manual, non-subscription rule with grouping `NONE` .
- The two locked rules (`BNL-TAPE-SE` , `KEK-RAW-SE`) pin replicas at those sites and cannot be deleted until the lock is released.

Want me to check where the replicas actually landed (e.g. where the `ANY=true` rule placed its copy)?

2 · FOLLOW-UP: VALIDATED IN BELLERAWDIRAC?

You

and is this datablock validation is completed in bellerawdairac ?

b2-computing-agent

Yes — in BelleRawDIRAC, datablock `/belle/Raw/e0042/physics/r01976/sub00` (id **41187**) is fully **Done**:

- **Status:** Done (last updated 2026-06-27 03:48:11)
- **Files:** 24, all accounted for
- **Parent dataset** `/belle/Raw/e0042/physics/r01976` (id 40859) is also marked Done

So the datablock has completed its full lifecycle (registration → replication → validation) in BelleRawDIRAC. This is consistent with the Rucio rules you saw earlier — all 5 rules in state OK with 24/24 locks satisfied.

One clarification: `/belle/Raw/e0042/physics/r01976/sub00` is the **datablock** name; the dataset is `/belle/Raw/e0042/physics/r01976` .

Want me to verify the per-file replication status (all 24 files Replicated) as a final cross-check?

WHAT THIS SHOWS

Just ask in plain English. No Rucio commands or SQL to remember — the agent queries Rucio and the BelleRawDIRAC database live and answers with what they report right now.

The follow-up switches systems, and the agent ties the two answers together.

Scenario: a user wants to find their datasets

1 · LIST A USER'S DATASETS

You

list the datasets of user mattb in rucio

b2-computing-agent

Datasets in scope `user.mattb`

There are **7 datasets** in scope `user.mattb` (complete list, not truncated). All are steering/output datasets, each containing exactly **1 file**:

#	DATASET (SCOPE:NAME)	FILES	FILE	SIZE
1	`user.mattb:/belle/user/mattb/mattb_55th_b2gm_example1/steering`	1	`example-mattb_55th_b2gm_example1.py`	2,218 B
2	`user.mattb:/belle/user/mattb/mattb_55th_b2gm_example2/steering`	1	`example-mattb_55th_b2gm_example2.py`	2,218 B
3	`user.mattb:/belle/user/mattb/mattb_55b2gm_Bd2JpsiKs_20261021/steering`	1	`Reconstruct_Bd2JpsiKS_template-mattb_55b2gm_Bd2JpsiKs_20261021.py`	3,253 B
… rows 4–6 omitted …				
7	`user.mattb:/belle/user/mattb/mattb_55b2gm_Bd2JpsiKs_20261021c/sub00`	1	`Bd2JpsiKS_00000_job590453880_00.root`	13,764 B

2 · WHERE ARE THE COPIES?

where are the replicas of /belle/user/mattb/mattb_55b2gm_Bd2JpsiKs_20261021c?

‹ 2/2 ›

b2-computing-agent

Replicas of `/belle/user/mattb/mattb_55b2gm_Bd2JpsiKs_20261021c`

This is a **container** (not a single dataset) holding 2 datasets / 2 files. Each file lives on exactly **one RSE**:

DATASET	FILE	RSE	STATE	SIZE
`…/steering`	`Reconstruct_Bd2JpsiKS_template-mattb_55b2gm_Bd2JpsiKs_20261021c.py`	`KEK-DISK-TMP-SE`	AVAILABLE	3,307 B
`…/sub00`	`Bd2JpsiKS_00000_job590453880_00.root`	`DESY-TMP-SE`	AVAILABLE	13,764 B

… PFNs omitted …

⚠ **Worth noting:** both replicas are on **TMP** RSEs (`KEK-DISK-TMP-SE` , `DESY-TMP-SE`), which are typically subject to cleanup policies — the files can disappear without a replication rule in place. If you want them to persist or be copied elsewhere, the next thing to check is the DID's rules. Want me to look up the rules attached to this container?

WHAT THIS SHOWS

Plain words in — the agent resolves the Rucio scope itself (**user.mattb**) and answers with live data from Rucio.

Follow-up conversation: replicating the dataset

1 · ASKED LOOSELY

You

how do I replicate this dataset to another SE?

b2-computing-agent

The documentation doesn't give a concrete command for replicating a dataset to another SE. Here's what it does say, verbatim:

⋯ quoted docs answer omitted ⋯

So the docs confirm replication is done via the `gb2` tools, but don't spell out the exact command or flags. I don't have a grounded answer for the precise syntax, so I won't invent one.

Honest, but a miss — the docs do have a **Replica management** section; the search didn't surface it.

2 · ASKED DIRECTLY

You

Which gbasf2 command replicates a dataset?

< 2/2 >

b2-computing-agent

The command is `gb2_ds_rep` [3].

You specify the destination SE with the `-d` flag, followed by the project name or the full LPN of the dataset [3].

⋯ two generic examples omitted ⋯

So for your case, that would look like:

console — Collapse Save Copy

```
1 $ gb2_ds_rep -d <destination_SE> /belle/user/mattb/mattb_55b2gm_Bd2JpsiKs_20261021
```

Sources [1] datasetSearcher.rst — Searching Datasets for Analysis > Datasets and Datablocks [2] index.rst — GBasf2 Documentation [3] dataManagement.rst — Data Management > Replica management

✓ **gb2_ds_rep**, cited to Replica management — matches the gbasf2 docs.

3 · THEN CHECKED LIVE IN RUCIO

You

can you check if any SE in India this can file be replicated ?

< 2/2 >

b2-computing-agent

India SEs — replication status

There are **2 RSEs in India**, both at TIFR, but **neither is currently available for replication**:

RSE	TYPE	READ	WRITE	STATUS
`TIFR-DATA-SE`	DISK	×	×	offline (availability = 0)
`TIFR-TMP-SE`	DISK	×	×	offline (availability = 0)

Bottom line: No, this file **cannot** currently be replicated to an SE in India. Both Indian RSEs are registered but offline — read and write are both disabled, so `gb2_ds_rep -d TIFR-DATA-SE` (or `TIFR-TMP-SE`) would not succeed right now.

✓ Both Indian SEs looked up live — offline, so it says the replication would not succeed instead of naming a target from memory.

THE LESSON

It refused to guess, cited its source, and checked Rucio live — the behaviour we want.

But this was a **context change — from Rucio to the gbasf2 docs** — and that switch still needs refinement and improvement: the wording of the question decides whether the right passage is found. The next fix is the search, not the model.

Next: one tool for expert shifters, one for users

Both are built from pieces you have already seen.

DATA PRODUCTION · EXPERT SHIFTERS

Data production expert shifter assistant

Reads the GitLab tickets and checks Rucio for the data behind them — and, in future, DiracX for the jobs.

TODAY	The expert shifter reads each ticket, then looks the problem up in Rucio and DIRAC by hand.
WITH THE ASSISTANT	One question: the agent reads the open tickets, checks the datasets in Rucio, and says what needs attention.

Built from the operations agent on slide 03, with GitLab and Rucio tools — DiracX next.

PLANNED

GBASF2 · FOR USERS · QUINN CAMPAGNA & PAUL GEBELINE

gb2_diagnostic --useAI

The gb2_diagnostic CLI users already run, with an AI explanation layered on.

TODAY	gb2_diagnostic checks failed jobs, jobs stuck waiting and failed downloads; the output is pasted into a user-support ticket for an expert to read.
WITH --USEAI	The same output, explained in plain words, with the matching fix from the gbasf2 docs.

Built from the docs assistant on slide 04, through the shared LLM service.

In development by the US team — planned to use the **KEK GPU resources for LLM inference**.

IN DEVELOPMENT

gb2_diagnostic with AI mode

Example from the development version (US team): a job stuck because it asks for too much CPU time.

TERMINAL · GBASF2

```
$ gb2_diagnostic --failed_job 1517217 --useAI
```

DIAGNOSIS

⚠ **CPUTime MISMATCH** (primary blocker):
Job requests CPUTime=500,000, which exceeds the maximum queue CPUTime at 6 out of 7 sites with known limits.

→ **IMMEDIATE ACTION** — reduce CPUTime parameter when resubmitting the job. Based on the docs, the formula is: **cputime = 20 × (total runtime on KEKCC in min).**
Use the gbasf2 option **--cputime <min>** or **--evtpersec <events_per_second>** on resubmit.
Kill the stuck job: gb2_job_kill 1517217, then resubmit with a corrected --cputime value.

1. **You add one flag**
   the same gb2_diagnostic command, plus **--useAI**
2. **It collects the facts**
   the job's parameters and the site limits from BelleDIRAC
3. **Sends them for analysis**
   to the analysis service — no copy-paste into a ticket
4. **Docs + AI + a check**
   the CPU-time formula comes from the gbasf2 docs; the validator checks the answer
5. **You get the fix**
   the cause, the options to change, and the exact commands to run

On-premises LLMs at KEK

WHY SELF-HOSTED

01 Data stays at KEK

Everyone is concerned about sensitive and personal (PII) data — here it never leaves KEK.

02 Hardened, closed cluster

No inbound connections from outside — access is limited to KEK.

03 Fixed hardware

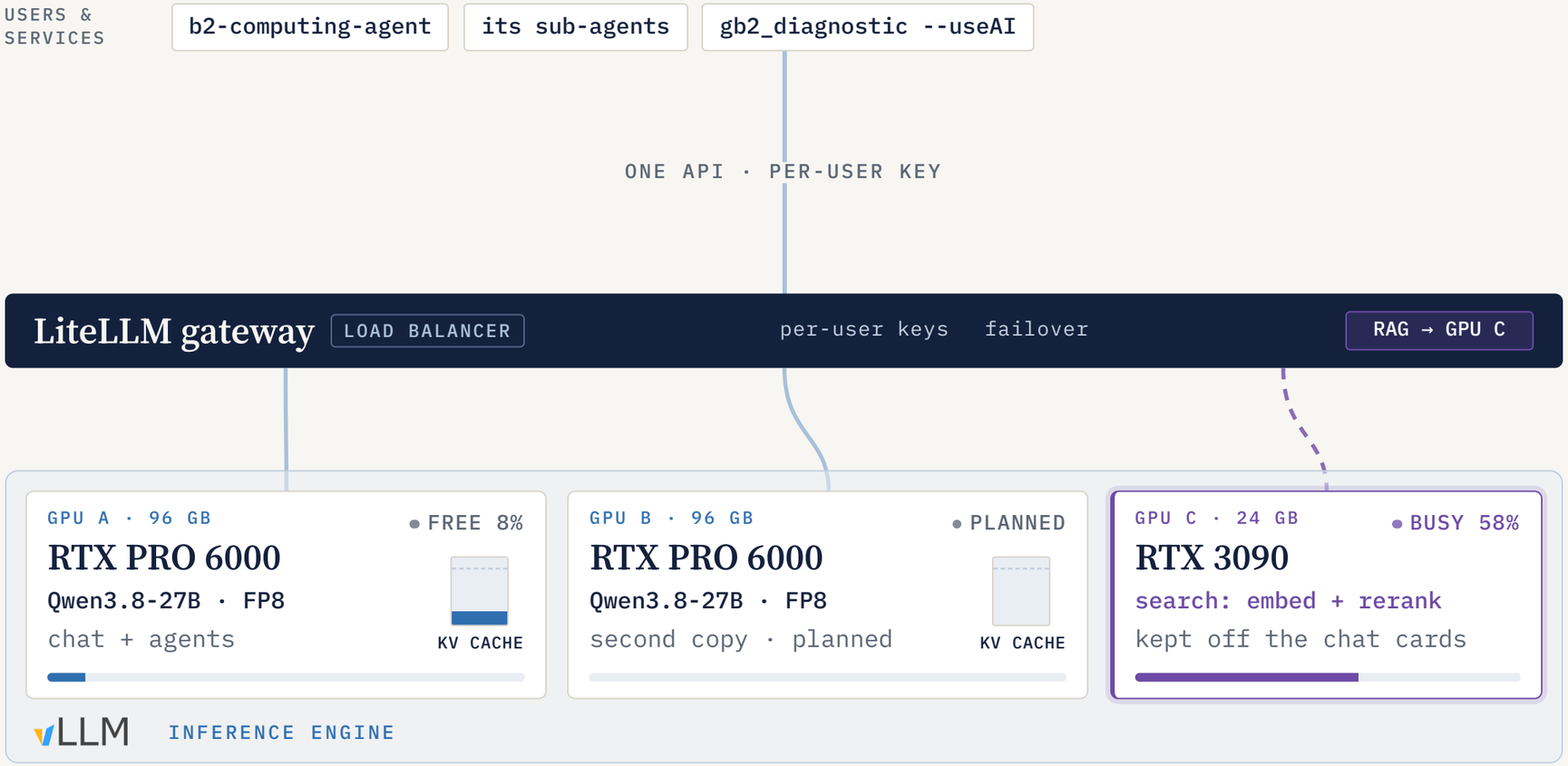
No cloud — a fixed set of GPU resources.

PILOT PHASE ON EXISTING GPUS

1. **The GPUs:** 2× RTX PRO 6000 Blackwell (96 GB each) plus an RTX 3090 for embedding and reranking, on the existing k3s cluster.
2. **Live now:** a pilot infrastructure, already used by a few users within KEK.
3. **Commissioning:** the newly acquired GPU resources are being commissioned — one new GPU is already running the pilot test while we validate its performance.
4. **Thanks to Ueda-san and Matsuoka-san** for their efforts on allocating the budget.

Hosting on Kubernetes: one gateway, three GPUs

One gateway sends every request to the right GPU.



Monitoring · Grafana · Langfuse

KEY FACTS

One address, one key

Every tool talks to the same API with its own key; usage is traced per key.

Search never slows chat

The 27B model has its own cards; embedding and reranking run on the RTX 3090.

Survives a GPU failure

With the second copy, the gateway fails over — and cards can be upgraded one at a time.

Grows by adding copies

Each model copy is its own Kubernetes service; a new GPU means one more copy.

55TH B2GM · KEK

Thank you

Questions?

dhirajk@post.kek.jp · dkalita@cern.ch

BACKUP

Details, if you want them

B1 The retrieval pipeline — hybrid search, reranker, relevance gate, corrective rewrite.

B2 gb2_diagnostic --useAI under the hood — API, docs search, AI model, validator.

B3 The hardware — the two big cards, memory, and what FP8 buys us.

B4 One service, many people — how the GPUs are shared.

B5 One address, two copies — routing and failover.

B6 How the KV cache works — the model's working memory.

B7 Follow-up questions — when the cache is reused, and when it misses.

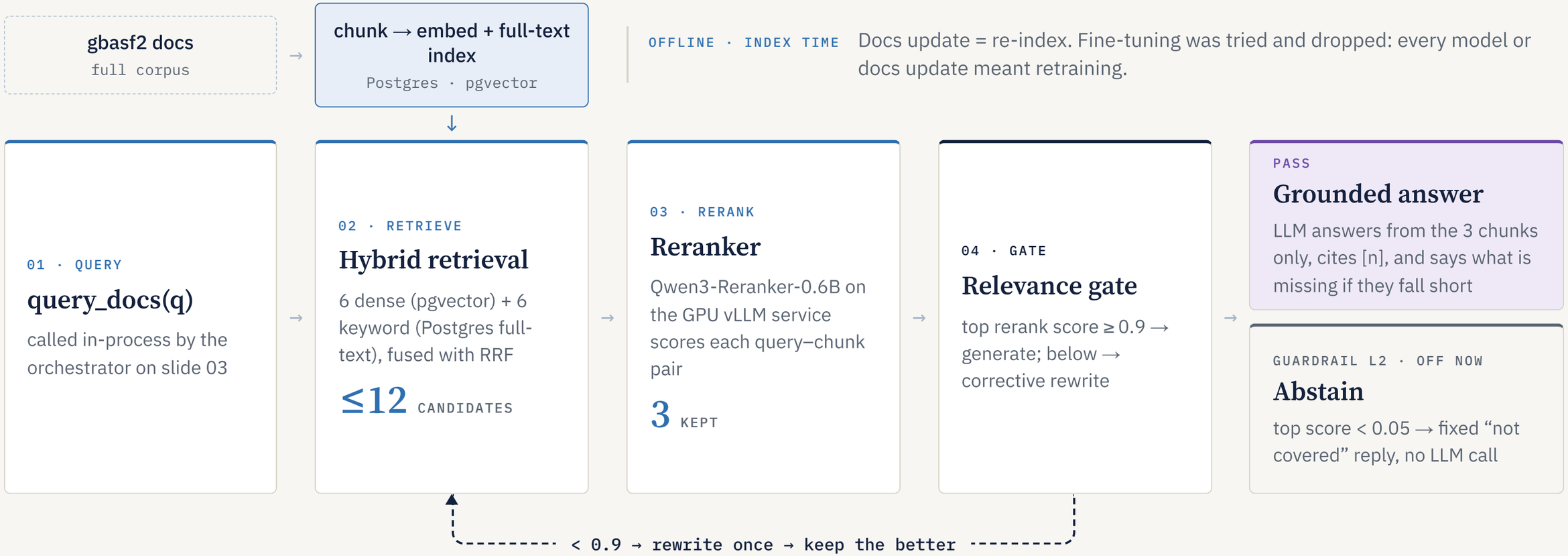
B8 Future considerations — LangGraph, Deep Agents, OpenClaw and Hermes Agent compared.

B9 Exploring next — decision models, RAFT and GraphRAG.

Same numbers as the talk · speaker notes included

Inside query_docs

Hybrid RAG over the gbasf2 corpus: dense + keyword retrieval, a cross-encoder reranker, a relevance gate with one corrective rewrite, and cited answers.



Guardrails off in the deployment (GUARD_ENABLED=0): the 0.05 floor was calibrated on bge-reranker, not Qwen3 — Re-tune before switching back on

gb2_diagnostic --useAI: how it works

The diagnostics users already collect, explained by an AI — with an anti-hallucination check.

DIAGNOSTIC COLLECTION

USER
User
reports a problem

⇄

BELLEDIRAC TOOL
gb2_diagnostic
runs the checks, sends the data

⇄

JOB MANAGEMENT
BelleDIRAC
job and transfer info

OBSERVABILITY
Langfuse
every request is traced

↓ diagnostic data

ANALYSIS PLATFORM

FASTAPI
REST API
receives the diagnostics

→

STEP 1
Data collector
gathers diagnostics + docs

→

STEP 2
Context builder
builds the enriched prompt

→

STEP 3
AI client
asks the model

→

STEP 4
Validator
anti-hallucination check

↕ query

RAG
Semantic search
over the gbasf2 docs

↕ request

AI MODELS · TODAY
Azure AI
planned: KEK GPU resources for LLM inference

Validated analysis returns through the API to gb2_diagnostic → **the user gets recommendations**

Two big cards, one helper

×2 · THE WORKHORSES

RTX PRO 6000 Blackwell

- 96 GB per card — a full 27B copy plus room for many users' conversations
- ~1.8 TB/s memory bandwidth — the real limit on answer speed
- native FP8 / FP4 · ECC memory · data-centre drivers for 24/7 service

×1 · THE HELPER

RTX 3090 · 24 GB

Embedder and reranker — one-pass jobs that need almost no KV cache. Kept on their own card so chat is never slowed by retrieval work.

PRECISION — BITS PER NUMBER

FORMAT	BITS	27B WEIGHTS	QUALITY
BF16 / FP16	16	~54 GB	reference
FP8	8	~27 GB	near-identical — **we serve this**
FP4	4	~14 GB	small loss — test first

Weights and working memory (KV cache) both run FP8 — roughly doubling the memory left for users on each card.

One service, many people

Nobody logs into a GPU. People use an application — gb2_diagnostic --useAI, the expert shifter assistant, raw-data monitoring — and the application shares the same service with everyone else.

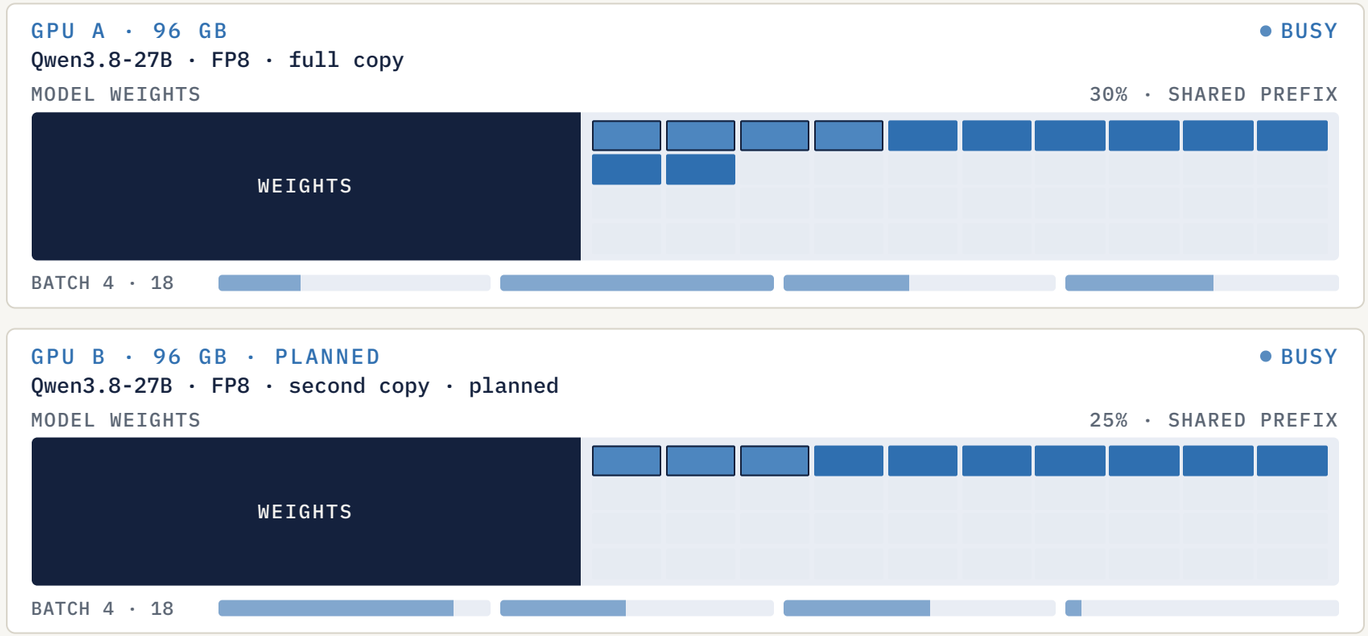
THEY COME FROM

- gb2_diagnostic --useAI
- expert shifter assistant
- raw-data monitoring

→ ONE SERVICE

EXAMPLE: PEOPLE — 20 talking

LONG INPUT — READ IN SLICES



WHY IT WORKS 05 / 05

01 **Everyone at once**
PagedAttention — memory is handed out in small pieces as each conversation grows.
Many more people at once

02 **Long conversations fill the memory**
FP8 memory — the same context kept in half the space.
About twice the context fits

03 **The same opening is read every time**
Prefix caching — the shared start of a question is processed once and reused.
Follow-ups start sooner

04 **One big upload freezes everyone**
Chunked prefill — a long input is read in slices between other people's work.
Answers keep streaming

05 **The GPU waits while it writes word by word**
Continuous batching — new questions join the running batch at every step.
More total throughput

One address, two copies

Your application knows one address. The service decides which GPU answers — and keeps your conversation on the same one.

WHAT IT MEANS FOR YOU

01 One address

Live — everything talks to the same name. Nobody needs to know which card is which.

02 Sent to the quieter card

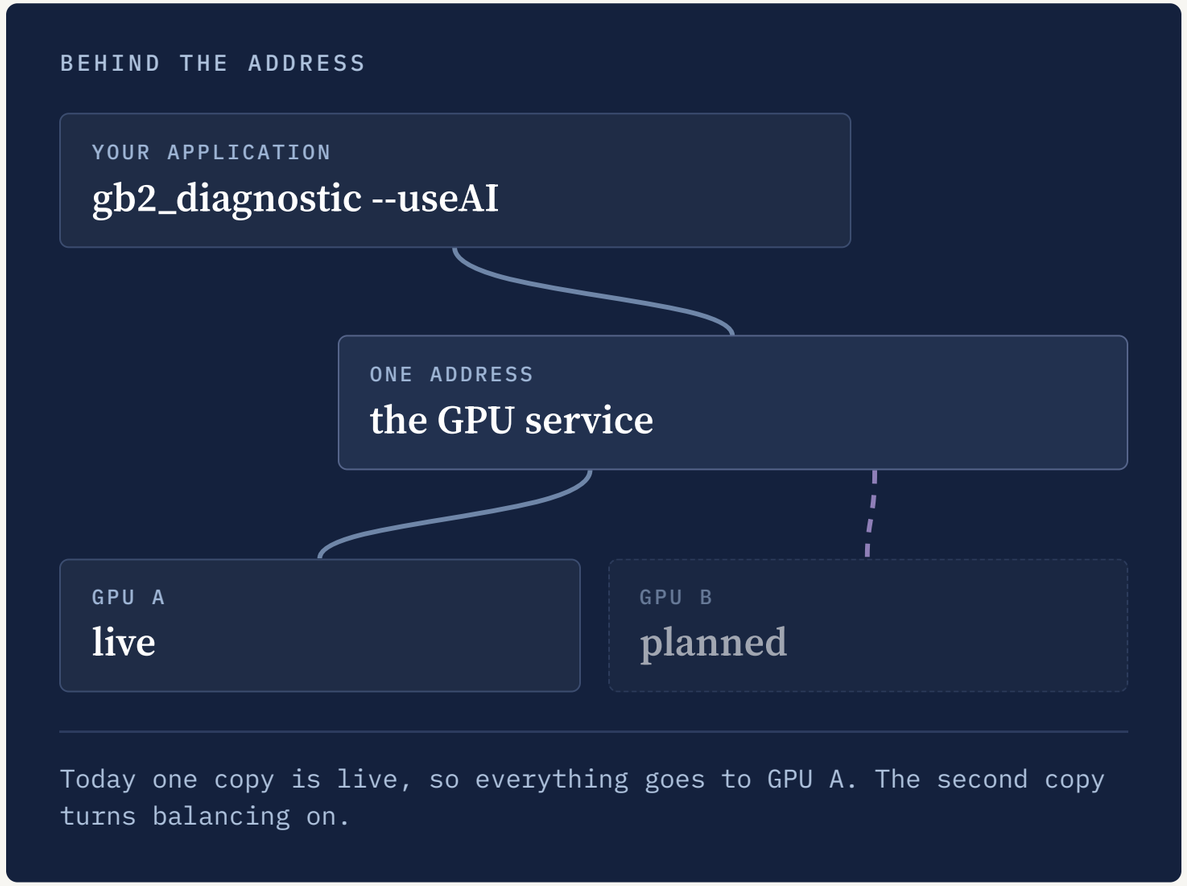
Planned with the second copy: your request goes to whichever GPU is less busy.

03 Your conversation stays put

Planned: follow-ups come back to the GPU that already holds your context, so answers start sooner.

04 Fair shares

Live: each application has its own key and limits, so one heavy job cannot fill the service.



How the KV cache works

The model's working memory: what it has already worked out about earlier tokens, stored so it is never recomputed.

ONE CONVERSATION

1 · Prefill — the whole prompt is read at once



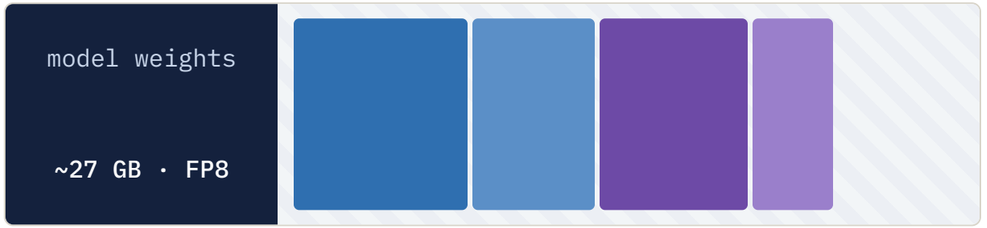
2 · Decode — the answer is written one token at a time



Every token's **keys** and **values** are computed once and kept. Each new token only computes its own — and reads the rest from the cache.

prompt, cached answer, cached

ONE 96 GB GPU



weights KV cache — shared by every open conversation

The space left after the weights decides **how many people** can be served at once — and **how long** their conversations can be.

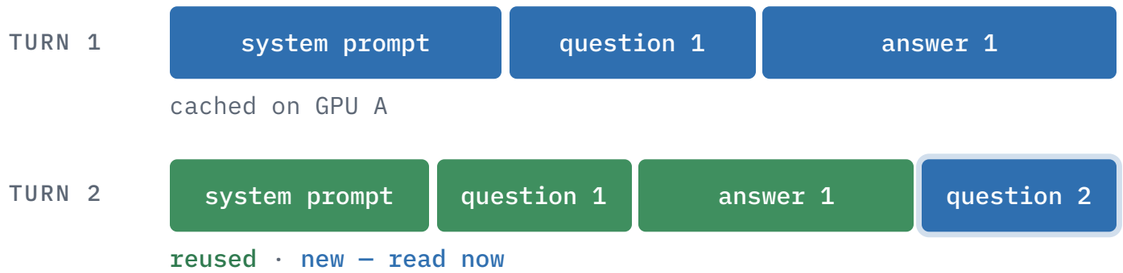
PagedAttention
memory in small blocks, taken only as a conversation grows

FP8 KV cache
half the memory per conversation

Prefix caching
a shared start is computed once and reused

Follow-up questions reuse the cache

The chat API is stateless: every follow-up sends the whole conversation again.



Only the new question is read → **the answer starts sooner**, even in a long conversation.

WHEN IT MISSES — THE WHOLE CONVERSATION IS READ AGAIN

1. **Sent to the other GPU**
   The cache lives on the card that served turn 1 — so with two copies, the gateway must keep a conversation on the same GPU.

2. **Evicted under load**
   On a busy card, the least-recently-used blocks are reclaimed for new requests.

3. **Something earlier changed**
   A timestamp in the system prompt, or rewritten history, breaks reuse from that point on.

For us: today there is one copy, so follow-ups hit the cache. With the second copy, the gateway needs sticky routing — and system prompts must stay identical across requests.

Next: from single answers to long-running tasks

	LangGraph TODAY	Deep Agents NEXT + NVIDIA OpenShell	OpenClaw	Hermes Agent
WHAT IT IS	low-level framework for stateful agents	an agent harness built on LangGraph	self-hosted personal assistant, driven from chat apps	self-improving personal agent (Nous Research)
PLANNING	we write the graph ourselves	built-in to-do planning	tasks and scheduled jobs	writes reusable skills from finished tasks
LONG TASKS	checkpoints; we manage the context	files as working memory, sub-agents in their own context, background runs	runs in the background, remembers across sessions	persistent memory and session search
FIT FOR COMPUTING	✓ runs b2-computing-agent	✓ same LangGraph runtime — our tools carry over; runs in an OpenShell sandbox with policy guardrails	✗ acts on the host by default (shell, files, tokens); sandbox is opt-in	~ separate runtime, built as a personal assistant

WHY DEEP AGENTS FOR LONG-RUNNING, AUTONOMOUS WORK

Plans and keeps going

e.g. “investigate every stuck datablock overnight and write a report”.

Reuses what we have

The Rucio, MySQL, FTS and k3s sub-agents, Langfuse tracing, read-only MCP servers.

Stays safe

Runs inside an NVIDIA OpenShell sandbox with policy guardrails, and asks a human before any change.

Models and methods we are exploring

Guiding principle: better answers without adding operational effort.

PLANNED

Decision models

A new kind of small model that gives a structured answer — yes / no, a probability or a ranking — instead of free text.

USE CASES FOR US

- route a question to the right sub-agent
- check a tool call is safe before it runs
- decide whether retrieved docs are good enough
- check a long-running task is really finished

TRIED · PARKED

RAFT

Retrieval-augmented fine-tuning: fine-tune the model on questions together with retrieved documents, so it learns to answer from them.

OUR EXPERIENCE

- results were good
- but every newer model means fine-tuning again
- that is ongoing effort we want to avoid — plain RAG keeps updates to a re-index

TESTING

GraphRAG

Build a graph of entities and their relations from the documents, then retrieve from the graph as well as from text passages.

WHY

- better data extraction across many pages
- answers that connect facts from different documents
- testing whether it improves our results